

App name

Zeno

The Vibe: A combination of "Zen" and "Zero." It represents zero friction, zero guilt, and zero mental clutter. It's sharp, modern, and has a very tech-forward Apple aesthetic.

The Complete "ZenAdmin" Package

If you build this, your app now handles:

1. **Time & Energy:** (Mood-Sync Routine Builder)
2. **Finances:** (Subscription Ghostbuster via Plaid/Gmail API)
3. **Mental Clutter:** (The "Brain Dump")
4. **Decision Fatigue:** (The Energy-Aware Meal Solver)
5. **Social Boundaries:** (The Social Battery Protector)

1. The "Brain Dump" (Audio-to-Action)

When people are overwhelmed, even typing out a to-do list is too much effort. They just need to vent their thoughts.

- **The Feature:** A simple, giant microphone button on the home screen. The user just talks out loud: *"I'm so stressed. I need to call my mom back, the dog needs his heartworm pill tomorrow, and I'm anxious about my presentation on Friday."*
- **The AI Magic:** Your Spring Boot server sends that transcript to Gemini. Gemini extracts the actionable items, categorizes them, and adds them to the **Tasks** table. It ignores the venting, but logs the "anxious" sentiment in the **MoodLogs** table.
- **React Native & Spring Boot Stack:** [@react-native-voice/voice](#) for capturing the audio transcript -> POST request to Spring Boot -> Spring Boot prompts Gemini to extract JSON tasks -> Saves to PostgreSQL.

2. The "Decision Fatigue" Meal Solver

"What's for dinner?" is one of the most exhausting daily questions, especially when energy is low.

- **The Feature:** A food module that ties directly into your Mood/Energy tracker.
- **The AI Magic:** If the app knows your battery is at 10%, it doesn't suggest a 45-minute cooking recipe. It looks at your saved preferences and says, *"You're exhausted. Just make a peanut butter sandwich or order from that Thai place you like. Here's a button to open UberEats."* If your energy is at 90%, it suggests a healthy recipe using ingredients you previously logged.

- **React Native & Spring Boot Stack:** React Native UI for the meal prompt -> Spring Boot fetches your current `energy_score` from PostgreSQL -> Spring Boot asks Gemini for an energy-appropriate meal suggestion.

3. The "Social Battery" Protector

Social obligations are a massive source of silent stress. People often agree to things because they don't have the mental energy to figure out how to say "no" politely.

- **The Feature:** A quick-action template generator for declining invitations, canceling plans, or pushing back deadlines without burning bridges.
- **The AI Magic:** The user pastes a text message or email they received (e.g., "Hey, can you help me move this weekend?"). They select a vibe: "Polite Decline," "Reschedule for Next Month," or "Professional Boundary." The AI instantly drafts a warm, empathetic response they can copy and paste.
- **React Native & Spring Boot Stack:** A simple text input in React Native -> Spring Boot sends the text and the "Vibe" parameter to the Gemini API -> Returns the drafted text to the phone.

Standard to-do apps (like Todoist or Apple Reminders) are "dumb." They just hold text. When a user is tired, seeing a huge list of text causes anxiety, so they close the app and do nothing. Your app prevents that anxiety by adapting to their human reality.

Time & Energy (Mood-Sync Routine Builder)

Here is the complete, simplified breakdown of exactly how the **Time & Energy (Mood-Sync Routine Builder)** works from start to finish.

Think of this feature as a highly empathetic personal assistant. Its only goal is to make sure you get things done without burning out.

Here is exactly what happens in every possible scenario.

Scenario 1: Getting Tasks INTO the App (The Setup)

Before the app can help, it needs to know what you have to do.

- **What the User Does:** You can either type a task and manually select a due date from a calendar, OR you just tap the microphone and say: "I need to finish the big work presentation by tomorrow at 5 PM."

- **What the App Does:** If you use your voice, the Spring Boot server sends the audio text to the AI. The AI automatically extracts the title ("Big Work Presentation") and the exact deadline (Tomorrow, 5 PM). It saves this perfectly organized into your database.
- **The Manual Override:** If the AI accidentally sets the deadline for *today* instead of *tomorrow*, you just tap the task, and a standard calendar pops up so you can manually fix it. The app always trusts you over the AI.

Scenario 2: The High Energy Day (The "Consent" Mode)

You wake up feeling rested, but the app doesn't just assume you want to suffer.

- **The Check-In:** You report 90% energy.
- **The App's Question:** Instead of just throwing "Clean the Garage" (High Effort) at you, the app pauses and asks: *"You have great energy today! You have a difficult task pending ('Clean the Garage'). Do you want to tackle this today, or keep it light?"*
- **The Result:** * If you tap **"Let's do it,"** the app puts it at the top of your list so you can crush it.
 - If you tap **"Keep it light,"** the app respects your boundaries, hides the garage task, and just gives you medium/easy tasks for the day.

Scenario 3: The Low Energy Day + No Urgent Deadlines ("Rest" Mode)

You are exhausted, but luckily, nothing major is due right away.

- **What the User Does:** You do the morning vibe check and say, *"I barely slept. I'm at 20% energy."*
- **What the App Does:** The AI looks at your list and sees big tasks like "Organize the attic," but notices they aren't due for weeks. It completely **hides** them from your screen to prevent anxiety. It only shows you 1 or 2 incredibly easy tasks, like "Water the plants" or "Take out the trash."

Scenario 4: The Low Energy Day + URGENT Deadline ("Micro-Stepping" Mode)

This is the magic scenario we discussed. You are exhausted, but you have a massive task that *must* be done tomorrow.

- **What the User Does:** You report 20% energy. You expect the app to show you the terrifying "Finish Big Work Presentation" task, which usually makes you want to quit and go back to sleep.

- **What the App Does:** The AI realizes it can't hide this task because of the deadline. Instead, it uses **Micro-Stepping**. It transforms the main task into an "Accordion" dropdown.
- **The Result:** You don't see "Finish Presentation." You see a gentle message and three tiny checkboxes:
 - ☐ *Step 1: Open PowerPoint and name the file.*
 - ☐ *Step 2: Write 3 bullet points for the intro.*
 - ☐ *Step 3: Find one image.*

As you check off these tiny steps, a progress bar fills up. You build momentum without feeling overwhelmed.

Why this changes everything

In a normal app, the user is forced to adapt to a rigid list of text. In **ZenAdmin**, the list constantly adapts to the human being using it. It respects your limits while still ensuring you meet your responsibilities.

Finances (Subscription Ghostbuster)

This is the ultimate, finalized blueprint for the **Finances (Subscription Ghostbuster)** feature.

Because we want to keep this architecture completely free to run and ensure it gets approved by the App Store and Google Play, we are officially dropping Plaid and relying 100% on the **Gmail API**.

Here is exactly what happens in every possible scenario, combining the Google Login, the automated background workers, the AI, and the push notifications.

Scenario 1: The Initial Hookup (First-Time Setup)

Before the app can do any background work, the user has to grant permission securely.

- **The Action:** The user downloads the React Native app, creates an account, and taps a button saying "Connect Gmail."
- **The Secure Handshake:** A Google login screen appears. Google asks, "*ZenAdmin wants to view your email messages (Read-Only). Allow?*" The user taps **Allow**.
- **The Token Exchange:** Google gives your app a long-lasting "Refresh Token" (a master key). Your React Native app securely sends this token to your Spring Boot server. Spring Boot locks this master key safely in your PostgreSQL database next to the user's profile.

Scenario 2: The Silent Workers (The 00:00 & 18:00 Cron Job)

The user is living their life. The app is completely closed.

- **The Trigger:** At exactly midnight and 6:00 PM, your Spring Boot server wakes up using a `@Scheduled` background task.
- **The Smart Search:** Spring Boot uses the master key to get a temporary access pass from Google. It searches the user's Gmail *only* for new emails with subjects like "Receipt" or "Subscription" that arrived since the last check.
- **The AI Data Entry:** Spring Boot sends those messy receipt emails to the Gemini API. Gemini perfectly extracts the data (e.g., Service: Netflix, Cost: 1,500 LKR, Cycle: Monthly) and sends it back as JSON.
- **The Save:** Spring Boot saves these newly found subscriptions into your PostgreSQL `subscriptions` table.

Scenario 3: The Smart Alert & Storage (The Database Logic)

Spring Boot realizes a bill is coming up and needs to warn the user.

- **The Calculation:** Spring Boot scans the database and sees that Netflix is going to charge 1,500 LKR in 3 days. It also sees 2 other subscriptions marked as `UNREVIEWED`.
- **The Storage:** Before sending a notification to the phone, Spring Boot creates a permanent record in a new PostgreSQL `notifications` table:
 - `message`: *"Heads up! Netflix is going to charge you 1,500 LKR in 3 days. You also have 2 other active subscriptions you haven't reviewed recently."*
 - `status`: `UNREAD`
- **The First Ping:** Spring Boot instantly triggers Firebase to send this exact message to the user's phone as a push notification.

Scenario 4: The Persistent Nag (The Daily Loop)

The user is busy, sees the notification on their lock screen, and swipes it away without opening the app. They still haven't dealt with the impending charge.

- **The Next Check:** When the Spring Boot cron job runs again the next day, it checks the `notifications` table.
- **The Loop:** It sees that the 1,500 LKR Netflix warning is *still* marked as `UNREAD`.
- **The Repeat Ping:** Spring Boot tells Firebase to send the push notification again. It will persistently repeat this process every day, refusing to let the user accidentally lose money, until they finally open the app to deal with it.

Scenario 5: The Resolution & Action (Ghostbusting)

The user finally has a free minute and wants the warnings to stop.

- **The Acknowledgment:** The user taps the push notification. The React Native app opens.
 - **The Silent Update:** The moment the dashboard loads, React Native tells Spring Boot, *"The user saw it!"* Spring Boot immediately updates that notification in the database from **UNREAD** to **READ**. The daily nagging stops forever.
 - **The Ghostbusting:** The user looks at their dashboard, taps the Netflix subscription, and hits **"Cancel."** * **The Final Action:** * *Direct Link:* The app opens the phone's browser directly to the hidden "Confirm Cancellation" page.
 - *AI Draft:* If there is no direct link, Spring Boot tells Gemini to write a polite cancellation email. React Native opens the user's Gmail app with the email fully written, addressed, and ready to send.
 - **Route A (Direct Link):** If the app knows the exact cancellation URL for Adobe, it instantly opens the phone's web browser straight to the "Confirm Cancellation" page, bypassing the maze of settings menus.
 - **Route B (AI Email Draft):** If it's a smaller company that requires an email to cancel (like a local gym), Spring Boot asks Gemini to draft a formal cancellation email. React Native opens the user's Gmail app with the email fully written and addressed to the gym's support team. The user just presses "Send."
-

The Final Verdict

This is a complete, closed-loop system. It finds the data silently, stores the warning safely, nags the user until they pay attention, and then does the hard work of actually canceling the service for them

Mental Clutter (Brain Dump)

The **Mental Clutter (Brain Dump)** feature fixes this by acting as an empathetic, digital therapist. The user just talks, and the app does all the organizing.

Here is exactly how this feature works from start to finish.

Scenario 1: The Overflow (The User Input)

The user's brain is completely fried, and they have too much going on.

- **The Action:** The user opens the React Native app. Instead of tapping "Add Task," they press and hold a giant, friendly Microphone button on the home screen.
- **The Venting:** They just start talking, stream-of-consciousness: *"I am so stressed and exhausted today. I need to call my mom back, the dog needs his heartworm pill tomorrow morning, and I am completely panicking about my tax documents being due next week."*

Scenario 2: The Relay (Phone to Server)

The app captures this chaotic rant and prepares it for processing.

- **React Native:** Your app uses a library like `@react-native-voice/voice` to transcribe that spoken audio into a raw block of text in real-time.
- **The Handoff:** React Native takes that messy paragraph of text and sends it via an HTTP POST request to your Spring Boot server.

Scenario 3: The AI Sieve (The Gemini Magic)

This is where the heavy lifting happens. Your Spring Boot server acts as the manager, and Gemini acts as the data-entry clerk.

- **The Prompt:** Spring Boot sends the text to the Gemini API with strict instructions: *"Read this user's journal entry. Do two things: First, extract any actionable chores, figure out their deadlines, and guess how much effort they take. Second, analyze the user's emotional state and give me an energy score from 1 to 10."*
- **The Extraction:** Gemini filters out the "fluff" and returns a clean JSON object to Spring Boot. It identifies:
 - **Task 1:** Call Mom (Effort: Low, Deadline: None)
 - **Task 2:** Give dog heartworm pill (Effort: Low, Deadline: Tomorrow Morning)
 - **Task 3:** Prepare tax documents (Effort: High, Deadline: Next Week)
 - **Mood:** Stressed/Exhausted (Energy Score: 2/10)

Scenario 4: The Split Storage (PostgreSQL Routing)

Spring Boot receives this perfect JSON data and routes it to the exact right places in your database so the rest of your app can use it.

- **Routing the Chores:** Spring Boot inserts the three extracted tasks directly into your `tasks` table.
- **Routing the Mood:** Spring Boot inserts the "Stressed/Exhausted" vibe and the "2/10" energy score into your `mood_logs` table.
- **The Synergy:** *Because* Spring Boot saved that low energy score, it automatically triggers the **Time & Energy (Mood-Sync)** feature we built earlier!

Scenario 5: The Relief (The Dashboard Experience)

The user finishes speaking and lets go of the microphone button.

- **The Magic Reveal:** The messy block of text vanishes from their screen. In its place, the React Native app displays a beautifully organized dashboard.
- **The Empathy:** Because the app knows they are stressed (from the `mood_logs` table), it displays a gentle message: *"I've got this. Since you're feeling exhausted,*

I've hidden your taxes for now. Let's just focus on giving the dog his pill tomorrow. I've also added calling your mom to your low-effort list."

Why this changes the game

You have completely eliminated "Decision Fatigue." The user didn't have to decide what priority "Call Mom" was, or manually open a calendar to select "Tomorrow" for the dog's pill. The app listened, understood the context, organized the data, and adjusted their entire day to protect their mental health.

4. The "Passive Recovery" Engine (Replacing the Meal Solver)

Standard fitness trackers cause guilt by demanding 10,000 steps or manual calorie logging. ZenAdmin takes the opposite approach. It uses native phone sensors to silently measure the user's physical battery and actively protects them from burnout, doomscrolling, and desk fatigue.

This engine is broken down into four core systems, prioritizing human agency above all else.

A. The "Sleep-Debt Auto-Shield" (Hierarchy of Truth)

The app uses passive data to proactively protect the user, but always defers to their actual, self-reported energy level.

- **The Feature:** A background sync that checks the user's sleep data and *proposes* a daily workload adjustment, requiring zero manual entry to generate, but allowing a 1-click manual override.
- **The Hierarchy of Truth Logic:** Spring Boot treats explicit user input (manual mood logging) as Priority 1, and passive sensor data (HealthKit/Google Fit) as Priority 2.
- **The AI & UX Magic:** If the system detects the user only got 4 hours of sleep, it automatically prepares a low-effort schedule. However, it presents a choice: *"I see you had a rough night of sleep. I've prepared a 'Rest Mode' schedule for today, but how are you actually feeling?"* * **The Override:** Below the text are two buttons. If the user taps *"Actually, I feel great,"* the app instantly overrides the passive sleep data, logs a high energy score, and restores the normal, high-effort task list. Human agency always wins.

B. The "Brain Reset" / Dopamine Menu (Active Intervention)

When a user is exhausted, they default to doomscrolling, which drains them further. This feature intercepts that impulse with a curated, zero-friction recovery activity.

- **The Feature:** A highly visible "Brain Reset" button on the dashboard. When pressed, the app suggests exactly *one* personalized recovery activity based on their saved hobbies (e.g., 3D modeling, gaming, music production).

- **The AI Magic:** Spring Boot calculates the user's current `energy_score` and checks their calendar for free time. Gemini generates a tailored, time-boxed suggestion.
 - *High Energy (8/10): "You've got 45 mins! Try blocking out a quick low-poly prop in Blender." * Low Energy (2/10): "You're running on empty. Let's just do a 3-minute guided breathing exercise."*
- **The Swipe UX:** Presented as a Tinder-style card. If the user isn't feeling the suggestion, they swipe left, and Gemini instantly generates the next idea based on their profile.

C. The "Desk-Tension Interceptor" (Smart Reminders)

Prevents the physical burnout of hyper-focusing on a screen for too many hours straight.

- **The Feature:** A smart pedometer monitor that sends somatic empathy checks instead of generic "stand up" nags.
- **The AI Magic:** If the user's step count hasn't changed in 3 hours during the workday, the app knows they are locked at their desk. It sends a push notification: *"You've been locked in for 3 hours. Drop your shoulders, unclench your jaw, and look away from the screen for 60 seconds."*

D. The "Focus Shield" & Time-Awareness (Conflict Prevention)

An assistant shouldn't interrupt you when you are panicking about a deadline.

- **The Feature:** An automatic "Do Not Disturb" mode that protects the user's flow state and ensures recovery suggestions fit into their actual schedule.
- **The Time-Aware Logic:** When suggesting a "Brain Reset" hobby, the AI checks the `tasks` table. If the next task starts in 20 minutes, Gemini is strictly prompted to suggest an activity that takes *maximum* 10 minutes, displaying a live countdown timer on the screen so the user isn't late.
- **The Focus Shield Logic:** Before sending a "Desk-Tension" or "Mood Check" notification, Spring Boot checks if an `is_critical` task is happening within the next 2 hours. If true, all notifications are suppressed to protect the user's focus.

Tech Stack & Database Integration

- **React Native:** Uses `react-native-health` (iOS) and `react-native-google-fit` (Android) for background sensor checks.
 - **New Database Tables:**
 - `health_metrics`: Stores daily `sleep_hours` and `step_counts`.
 - `user_preferences`: Stores a JSON array of the user's hobbies for the Brain Reset AI prompt.
 - **Table Updates:** * `mood_logs` gets a new `data_source` column to distinguish between 'sensor' data and 'manual' overrides.
 - `tasks` gets an `is_critical` boolean flag to trigger the Focus Shield.
-

5. Predictive Health & Burnout Forecasting

Moving beyond daily survival, ZenAdmin acts as a long-term wellness analyst. It correlates the user's physical sensor data with their task completion and mood logs to find their personal "Golden Routine" and predict burnout before it strikes. Crucially, it does this without gamifying their health or inducing guilt over missed goals.

A. The "Burnout Forecaster" (Predictive Analytics)

The app identifies dangerous trends by looking at the intersection of sleep debt, workload, and declining mood.

- **The Logic:** Every Sunday night, a background job analyzes the past 7 days of health metrics, mood logs, and the upcoming week's task load.
- **The AI Magic:** The system looks for the "Burnout Formula" (e.g., Declining Sleep + High Upcoming Effort = Crash). Gemini generates a proactive warning for Monday morning to intercept the crash.
- **The UX:** Instead of a scary red graph, the user gets an empathetic, actionable alert: *"You've averaged 5 hours of sleep for the last 4 days, and you have 6 high-effort tasks scheduled for Tuesday. You are on track for a mid-week crash. I strongly recommend we move 3 of these tasks to Thursday. Tap here to auto-reschedule."*

B. The "Golden Routine" Discovery (Positive Correlation)

Users often don't know what actually makes them productive. The AI finds the hidden patterns in their data to highlight what works.

- **The Logic:** The database constantly cross-references days with high task completion rates against the user's health metrics and "Brain Reset" activities.
- **The AI Magic:** Gemini translates raw data points into actionable, positive lifestyle insights.
- **The UX:** The app occasionally drops a positive "Aha!" moment on the dashboard: *"Did you know? On days where you get at least 7 hours of sleep and use the 'Brain Reset' for a 15-minute gaming session, your afternoon energy never drops below an 8/10, and you complete 40% more tasks."*

C. The "Mind & Body" Weekly Report (Guilt-Free Visualization)

A weekly summary that visualizes progress without gamifying it into a high-pressure competition.

- **The Logic:** A dedicated "Insights" tab in the app that focuses on trends rather than strict daily quotas.
- **The UX:** Uses clean, minimalist line charts to plot 'Average Sleep' overlaid with 'Average Energy Score'.
- **The AI Context:** Because raw graphs can cause anxiety, Gemini always provides a gentle, contextual summary below the chart: *"Your sleep was a bit erratic this week,*

which explains the low energy on Thursday. But you did a great job protecting your boundaries and keeping your task load light when you needed it."

Tech Stack & Database Architecture

- **React Native Frontend:** Uses charting libraries like `react-native-chart-kit` or `victory-native` to render the clean, iOS-style minimalist graphs.
- **New Database Table: `health_insights`**
 - `id` (BIGSERIAL, Primary Key)
 - `user_id` (UUID, Foreign Key)
 - `insight_type` (VARCHAR): e.g., 'BURNOUT_WARNING', 'GOLDEN_ROUTINE'.
 - `insight_text` (TEXT): The AI-generated message.
 - `generated_at` (TIMESTAMP)
- **Backend Processing:** Spring Boot uses a `@Scheduled(cron = "0 0 2 * * SUN")` job to run the heavy data analysis at 2:00 AM every Sunday. It executes complex SQL joins across `health_metrics`, `tasks`, and `mood_logs`, passing the summarized JSON to Gemini for insight generation so the app is fast and ready when the user wakes up.

Social Battery Protector

The **Social Battery Protector** acts as your personal PR manager. It takes the emotional labor out of setting boundaries.

Here is exactly how this feature works from the moment you get an overwhelming text to the moment you hit send.

Scenario 1: The Incoming Guilt Trip (The Trigger)

You are exhausted, lying on the couch, and your phone buzzes.

- **The Message:** A friend or coworker texts you: *"Hey! A bunch of us are going out for drinks tonight and then heading to a party. You have to come!"* or *"Can you look over this massive project for me this weekend?"*
- **The Panic:** Your social battery is completely empty. You know you need to say no, but staring at the blank keyboard makes you feel guilty.

Scenario 2: The Shield Activation (React Native UI)

Instead of agonizing over the keyboard for 20 minutes, you let ZenAdmin handle the diplomacy.

- **The Action:** You copy the message you just received and open your React Native app. You navigate to the "Social Shield" tab and paste the message into a text box.
- **The Vibe Check:** Below the text box, you see a row of "Boundary Buttons." You just tap the one that matches your goal:
 -  **"Hard Pass (Polite but Firm)"**
 -  **"Reschedule (Ask for next week)"**
 -  **"Professional Boundary (For coworkers)"**
 -  **"Soft Decline (For close friends/family)"**

Scenario 3: The Diplomat (Spring Boot + Gemini AI)

Your phone sends that data to your server so the AI can do the heavy lifting.

- **The Handoff:** React Native makes a quick **POST** request to Spring Boot containing the pasted message and the "Vibe" you selected (e.g., Soft Decline).
- **The AI Prompt:** Spring Boot sends this to the Gemini API with a strict instruction: *"You are an empathetic communication coach. The user received this text: '[Pasted Text]'. They need to say no using a 'Soft Decline' approach. Draft a short, warm, natural-sounding text message that protects their energy, says no clearly, but keeps the friendship strong. Do not over-apologize."*
- **The Generation:** Gemini instantly writes the perfect response.

Scenario 4: The Clean Getaway (The Output & Action)

Spring Boot sends the drafted text back down to your phone in milliseconds.

- **The Reveal:** On your screen, a beautiful, stress-free message appears: *"Hey! That sounds like so much fun, but I am completely running on empty today and need to recharge at home tonight. Have the best time, and let's catch up soon!"*
- **The 1-Click Send:** Underneath the text, there is a giant **"Copy & Open WhatsApp"** (or Messages) button.
- **The Result:** You tap it, paste the text, hit send, and immediately go back to relaxing.

Why this completes the "ZenAdmin" ecosystem

If your app only helps people manage their chores and their money, it's only solving half the problem. Other people are one of the biggest drains on our time and energy. By adding this feature, your app literally protects the user's peace from all angles.

Additionally i think to create mobile simple game for the stress users to play (with multiplay)
In the same app its better

1. **users** Table

The central hub for authentication and background worker access.

Column Name	Data Type	Constraints	Purpose
id	UUID	Primary Key	Unique identifier for the user.
email	VARCHAR(255)	Unique, Not Null	The user's login email.
password_hash	VARCHAR(255)	Not Null	Securely hashed password.
gmail_refresh_token	VARCHAR(500)	Nullable	The master key Google gives your app to scan for receipts.
created_at	TIMESTAMP	Default NOW()	Account creation date.

2. **tasks** Table

This table powers the Mood-Sync Routine Builder. It supports the AI extracting the chores and handles the parent-child relationships required for "Micro-Stepping".

Column Name	Data Type	Constraints	Purpose
id	BIGSERIAL	Primary Key	Unique task identifier.

user_id	UUID	Foreign Key (users.id)	Links task to the user.
title	VARCHAR(255)	Not Null	e.g., "Big Work Presentation".
effort_level	VARCHAR(50)	Nullable	E.g., "Low" or "High".
deadline	TIMESTAMP	Nullable	Exact due date extracted by Gemini.
is_critical	BOOLEAN	Default FALSE	Triggers the "Focus Shield" to suppress notifications.
status	VARCHAR(50)	Default 'PENDING'	Tracks completion.
parent_task_id	BIGINT	Foreign Key (tasks.id)	Links tiny checkboxes to a main "Accordion" dropdown task.

3. [mood_logs](#) Table

The "brain" of the app. It powers the Time & Energy features and gives Gemini the context needed to adjust the user's day.

Column Name	Data Type	Constraints	Purpose
id	BIGSERIAL	Primary Key	Unique log identifier.
user_id	UUID	Foreign Key (users.id)	Links log to the user.

energy_score	INT	Check (1 to 10)	E.g., 2/10 for exhausted.
sentiment	VARCHAR(255)	Nullable	E.g., "anxious".
data_source	VARCHAR(50)	Not Null	Distinguishes between 'sensor' passive data and 'manual' overrides.
logged_at	TIMESTAMP	Default NOW()	When the mood was recorded.

4. **health_metrics** Table

This table silently stores the background data pulled from Apple HealthKit or Google Fit to power the Sleep-Debt Auto-Shield.

Column Name	Data Type	Constraints	Purpose
id	BIGSERIAL	Primary Key	Unique metric identifier.
user_id	UUID	Foreign Key (users.id)	Links metrics to the user.
sleep_hours	DECIMAL(4,2)	Nullable	Daily sleep hours.
step_counts	INT	Nullable	Daily step count.
logged_date	DATE	Not Null	The specific day these metrics belong to.

5. **user_preferences** Table

Provides the context needed for the "Brain Reset" menu so the app suggests activities the user actually cares about.

Column Name	Data Type	Constraints	Purpose
id	BIGSERIAL	Primary Key	Unique preference identifier.
user_id	UUID	Foreign Key (users.id)	Links preferences to the user.
hobbies	JSONB	Nullable	E.g., ["3D modeling in Blender", "Music production in FL Studio", "Gaming (Need for Speed)", "Learning Turkish"].

6. health_insights Table

Stores the predictive analytics and positive correlations generated by the Sunday night cron job.

Column Name	Data Type	Constraints	Purpose
id	BIGSERIAL	Primary Key	Unique insight identifier.
user_id	UUID	Foreign Key (users.id)	Links insight to the user.
insight_type	VARCHAR(50)	Not Null	E.g., 'BURNOUT_WARNING', 'GOLDEN_ROUTINE'.
insight_text	TEXT	Not Null	The AI-generated message.

generated_at	TIMESTAMP	Default NOW()	When the insight was created.
--------------	-----------	---------------	-------------------------------

7. **subscriptions** Table

Holds the recurring costs extracted by the Gemini API from Gmail receipts.

Column Name	Data Type	Constraints	Purpose
id	BIGSERIAL	Primary Key	Unique subscription identifier.
user_id	UUID	Foreign Key (users.id)	Links subscription to the user.
service_name	VARCHAR(100)	Not Null	E.g., Netflix.
cost	DECIMAL(10,2)	Not Null	E.g., 1500.
currency	VARCHAR(10)	Default 'LKR'	The currency of the charge.
billing_cycle	VARCHAR(50)	Not Null	E.g., Monthly.
status	VARCHAR(50)	Default 'UNREVIEWED'	"UNREVIEWED", "ACTIVE", or "CANCELLED".

8. **notifications** Table

Powers the Persistent Nag Loop, holding the alert until the user successfully resolves it.

Column Name	Data Type	Constraints	Purpose
-------------	-----------	-------------	---------

id	BIGSERIAL	Primary Key	Unique notification identifier.
user_id	UUID	Foreign Key (users.id)	Links alert to the user.
message	TEXT	Not Null	E.g., "Heads up! Netflix is going to charge you...".
status	VARCHAR(20)	Default 'UNREAD'	Keeps the daily loop active until marked 'READ'.
created_at	TIMESTAMP	Default NOW()	When the warning was generated.